AJAX / Fetch, JSON y carga dinámica

1. Cliente-servidor en Web-Apps: HTTP, recursos y APIs
2. AJAX: de XMLHttpRequest a Fetch
3. JSON: formato, validación y modelado de datos
4. Fetch en profundidad: respuestas, status, headers, errores
5. Estados de UI y arquitectura asíncrona
6. CORS, caching, rate limits y buenas prácticas
7. Mini-caso: catálogo que carga dataset remoto y gestiona fallos

Modelo cliente-servidor: recordatorio

- Cliente (browser): UI + lógica + fetch
- Servidor: recursos, datos, endpoints

Conceptos:

- recurso (`/data/items.json`)
- endpoint (`/api/items`)
- request/response
- latencia e incertidumbre

HTTP

- Métodos: GET, POST, PUT/PATCH, DELETE (en este curso usamos sobre todo GET)
- Status codes:
  - 2xx OK
  - 3xx redirect
  - 4xx error cliente (404, 401)
  - 5xx error servidor
- Headers (Content-Type, Cache-Control)

Ingeniería: leer status codes y actuar en consecuencia.

Qué es HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo que permite la comunicación entre cliente (navegador) y servidor. Cada interacción sigue el modelo:

```
Request  →  Response
```

El navegador envía una petición.
El servidor devuelve una respuesta.

Métodos HTTP

Los métodos indican **qué intención tiene la petición**.

GET

- Solicita datos
- No debería modificar recursos
- Es el más usado en este curso

Ejemplo:
```
fetch("/eventos.json")
```
Características: Cacheable, Visible en la URL

POST

- Envía datos al servidor
- Crea recursos

Ejemplo conceptual: Enviar formulario de registro.

PUT / PATCH

- Modifican recursos existentes
- PUT suele reemplazar
- PATCH modifica parcialmente

En este curso no se usan habitualmente.

DELETE

- Elimina un recurso del servidor

Status Codes (Códigos de estado)

Cuando el servidor responde, devuelve un número que indica el resultado.
Formato:
```
HTTP/1.1 200 OK
```

2xx — Éxito

Código	Significado
200	OK
201	Creado
204	Sin contenido

Ejemplo común:
```
response.ok  // true si 200-299
```

3xx — Redirección

Código	Significado
301	Redirección permanente
302	Redirección temporal

El navegador suele gestionarlos automáticamente.

4xx — Error del cliente

Código	Significado
400	Bad Request
401	No autorizado
403	Prohibido
404	No encontrado

Ejemplo típico en práctica:

```
fetch("/datos-inexistentes.json")
```

Devuelve 404.

5xx — Error del servidor

Código	Significado
500	Error interno
502	Bad Gateway
503	Servicio no disponible

Aquí el problema no está en el cliente.

Headers HTTP

Los headers contienen metadatos sobre la petición o respuesta.

Content-Type

Indica el tipo de contenido.
Ejemplo:

```
Content-Type: application/json
```

Permite al navegador saber cómo interpretar la respuesta.

Cache-Control

Controla la caché.
Ejemplo:

```
Cache-Control: max-age=3600
```

Indica cuánto tiempo se puede almacenar en caché.

Ejemplo completo con fetch

```
const response = await fetch("/eventos.json");

if (!response.ok) {
  throw new Error(`Error HTTP ${response.status}`);
}

const data = await response.json();
```

Aquí:

- Se comprueba el status code
- Se actúa en consecuencia
- No se asume que todo es 200

`fetch()` NO lanza error en 404 o 500.
Solo lanza error si:

- No hay conexión
- Hay fallo de red

Por eso siempre se debe validar:
`if (!response.ok)`

En la práctica

Aunque trabajéis principalmente con GET:
Debéis:

- Leer el status code
- Validar `response.ok`
- Gestionar errores 404
- Mostrar mensaje adecuado en UI

Eso forma parte de:

- Programación JS estructurada
- Gestión de estados
- UX profesional

HTTP no es solo “pedir datos”.
Es un protocolo con:

- Métodos (intención)
- Status codes (resultado)
- Headers (metadatos)

Caché HTTP y rendimiento en Web-Apps

La caché HTTP permite evitar descargar el mismo recurso múltiples veces.

Impacta directamente en:

- Tiempo de carga
- Consumo de red
- Experiencia de usuario
- Escalabilidad

¿Qué es la caché HTTP?

Cuando el navegador recibe una respuesta, puede almacenarla temporalmente.

En siguientes peticiones:

- Puede reutilizarla
- O validar si sigue siendo válida

Esto depende de los **headers HTTP**.

Header Cache-Control

Ejemplo:

Cache-Control: max-age=3600

Significa:

- El recurso es válido durante 3600 segundos
- El navegador no volverá a pedirlo al servidor

Muy útil para:

- JSON estáticos
- Imágenes
- CSS
- JS

Otros valores importantes

Cache-Control: no-cache

Obliga a revalidar con el servidor.

Cache-Control: no-store

No se guarda en caché.

ETag y validación condicional

Cuando el servidor responde, puede enviar:

ETag: "abc123"

En la siguiente petición, el navegador envía:

If-None-Match: "abc123"

Si el recurso no ha cambiado:

→ El servidor responde:

304 Not Modified

Sin enviar el cuerpo.

Esto ahorra ancho de banda.

Ejemplo aplicado a la práctica

```
fetch("/eventos.json")
```

Si el archivo es estático y se cachea:

- Primera carga → descarga completa
- Segunda carga → respuesta instantánea desde caché

Resultado:

- Mejor experiencia
- Menor latencia
- Mejor puntuación Lighthouse

Caché y fetch

fetch() respeta la política de caché por defecto.

Pero se puede configurar:

```
fetch("/eventos.json", {
  cache: "no-store"
});
```

Opciones posibles:

- default
- no-store

- reload
- no-cache
- force-cache
- only-if-cached

Impacto real en rendimiento

Sin caché:

- Cada carga → petición completa
- Más tiempo
- Más uso red

Con caché:

- Menos peticiones
- Respuesta casi inmediata
- Mejor UX

En la práctica cada recurso debería clasificarse:

Recurso	¿Debe cachearse?
CSS	Sí
JS	Sí
Imágenes	Sí
JSON estático	Sí
Datos dinámicos	Depende

La caché HTTP es una herramienta de:

- Optimización
- Escalabilidad
- Profesionalización del proyecto

HTTPS y seguridad HTTPS cifra y autentica el servidor

- Evita manipulación de datos en tránsito
- Muchas APIs externas exigen HTTPS
- Mixed content (HTTPS + HTTP) rompe recursos

La publicación final debe ser HTTPS (GitHub Pages/hosting equivalente).

AJAX: qué significa realmente

AJAX = cargar datos **sin recargar** la página.

- antes: `XMLHttpRequest`
- ahora: `fetch()` + Promises

Concepto clave: asincronía coordinada con estado/UI.

Promesas

1) "Una promesa representa un resultado futuro"

Una **Promise** es un objeto que "promete" que **más adelante** tendrás:

- un **valor** (éxito), o
- un **error** (fallo)

No te da el resultado ahora, te da un **manejador** para reaccionar cuando esté.

2) Estados: pending → fulfilled o rejected

Una promesa tiene estados internos:

- **pending**: "en curso" (aún no hay resultado)
- **fulfilled**: terminó bien y tiene un valor
- **rejected**: terminó mal y tiene una razón/error

Reglas de ingeniería:

- Una promesa **solo se resuelve una vez**.
- Una vez que pasa a fulfilled o rejected, **no vuelve atrás**.
- "Resuelta" significa: *ya no está pending* (da igual si fue éxito o error).

Ejemplo:

```
const p = fetch("/data.json"); // normalmente empieza pending
// luego será fulfilled (respuesta) o rejected (fallo red, etc.)
```

3) then encadena

then() sirve para dos cosas:

1. **Recibir el valor** cuando la promesa se cumple
2. **Encadenar** otro paso (que puede devolver valor o otra promesa)

Ejemplo:

```
fetch("/data.json")
  .then(r => r.json())          // r.json() devuelve Promise
  .then(data => procesar(data)) // puede devolver valor o Promise
  .then(result => pintar(result));
```

Modelo mental:

then es como una cinta transportadora: cada paso recibe lo anterior y produce lo siguiente.

4) catch captura errores

catch() captura:

- rechazos de la promesa anterior
- **y también errores lanzados** dentro de cualquier then anterior

Ejemplo:

```
fetch("/data.json")
  .then(r => r.json())
  .then(data => {
    if (!data) throw new Error("Datos inválidos");
    return data;
  })
  .catch(err => {
    // Captura tanto fallos de fetch como el throw de arriba
    console.error(err);
  });
```

Regla importante:

Un throw dentro de un then equivale a “rechazar” la promesa del chain.

Y otra regla clave:

- Si en catch() devuelves un valor, **recuperas** el flujo (la cadena puede continuar como éxito).

5) finally para limpieza

finally() se ejecuta **siempre**, haya éxito o error.

Usos típicos:

- ocultar un spinner de “Loading”
- liberar recursos
- reactivar botones
- cerrar modales

Ejemplo:

```
btn.disabled = true;
spinner.show();

fetch("/data.json")
  .then(r => r.json())
  .then(data => render(data))
  .catch(err => showError(err))
  .finally(() => {
    spinner.hide();
    btn.disabled = false;
  });
```

Regla: finally() **no recibe** el valor ni el error (su objetivo es limpieza).

`async/await`: asincronía legible

1) await "pausa" la función async, pero no bloquea el hilo global

Ejemplo:

```
async function cargar() {
  const r = await fetch("/data.json");
  const data = await r.json();
  return data;
}
```

Lo que ocurre a nivel de ejecución:

- La función cargar() **empieza** en la Call Stack.
- Al llegar a await fetch(...):
  - **se suspende** la ejecución *solo de esa función*.
  - cargar() **devuelve inmediatamente** una **Promise** (implícita).
  - El resto de instrucciones (lo que va después del await) queda "programado" para continuar cuando la promesa se resuelva.

await es como decir: "continúa aquí cuando tengas el resultado", pero sin congelar el navegador. Por eso await **no es un sleep**; no detiene el hilo principal. Detiene *la continuación de esa función*.

2) El event loop sigue vivo

Mientras la función está "pausada" esperando:

- el navegador puede seguir:
  - procesando clicks y eventos
  - ejecutando otros callbacks
  - renderizando la UI (si la pila está libre)
  - atendiendo timers
  - resolviendo otras promesas

Ejemplo mental:

- Tu función está esperando red (fetch)
- Pero el usuario puede seguir haciendo scroll o pulsando botones (si tu código no bloquea con tareas pesadas)

3) Mejora claridad y facilita try/catch

Con Promesas "puras":

```
fetch("/data.json")
  .then(r => r.json())
  .then(data => render(data))
  .catch(err => mostrarError(err));
```

Con async/await:

```
async function cargar() {
  try {
    const r = await fetch("/data.json");
    if (!r.ok) throw new Error("HTTP " + r.status);

    const data = await r.json();
    render(data);

  } catch (err) {
    mostrarError(err);
  }
}
```

A nivel de ingeniería, async/await ayuda a que el código sea:

- **Lectura lineal** (top-down) como código síncrono.
- Manejo de errores **centralizado** con try/catch.
- más **claro**
- más **mantenible**
- más fácil de **depurar**
- más fácil de evaluar (estructura y control de errores)

Resumen:

- async = "esta función devuelve una Promise siempre"
- await = "pausa **esta función** hasta que la Promise se resuelva"
- el navegador **sigue funcionando**
- try/catch = manejo de errores profesional y centralizado

Perfecto 👍 Te lo dejo completamente en **castellano**, manteniendo el nivel técnico y estructura tipo transparencia.

Ejemplo completo con async/await

(JSON + estados de UI + render eficiente + delegation)

Cargar datos JSON (locales o externos), reflejar correctamente el estado en la interfaz y renderizar una lista de forma eficiente, manteniendo escalabilidad y buena arquitectura.

Gestión de estados de la interfaz

Estados típicos en una Web-App:

- **Idle**: aún no se ha iniciado ninguna acción
- **Loading**: cargando datos (spinner, botones desactivados)
- **Success**: datos cargados correctamente
- **Error**: mensaje informativo + opción de reintentar
- **Empty**: no hay resultados

Nota de usabilidad: Cada operación asíncrona debe tener una traducción visual coherente.

Ejemplo completo con async/await

```
const lista = document.getElementById("lista");
const spinner = document.getElementById("spinner");
const errorBox = document.getElementById("error");
const emptyBox = document.getElementById("empty");
const botonRecarga = document.getElementById("reload");

function setLoading(estado) {
  spinner.style.display = estado ? "block" : "none";
  botonRecarga.disabled = estado;
}

function setError(mensaje = "") {
  errorBox.textContent = mensaje;
  errorBox.style.display = mensaje ? "block" : "none";
}

function setEmpty(estado) {
  emptyBox.style.display = estado ? "block" : "none";
}

function limpiarLista() {
  lista.innerHTML = "";
}

function renderLista(items) {
  const fragment = document.createDocumentFragment();

  items.forEach(item => {
    const li = document.createElement("li");
    li.className = "item";
    li.dataset.id = item.id;
    li.textContent = item.nombre;
    fragment.appendChild(li);
  });

  limpiarLista();
  lista.appendChild(fragment);
}

async function cargarDatos() {
  try {
    setError("");
    setEmpty(false);
    setLoading(true);

    const respuesta = await fetch("/eventos.json");

    // Validación HTTP
    if (!respuesta.ok) {
      throw new Error(`Error HTTP ${respuesta.status}`);
    }

    const datos = await respuesta.json();

    // Validación del contrato de datos
    if (!Array.isArray(datos)) {
      throw new Error("Formato de datos incorrecto: se esperaba un array");
```

```
    }

    if (datos.length === 0) {
      limpiarLista();
      setEmpty(true);
      return;
    }

    renderLista(datos);

  } catch (error) {
    limpiarLista();
    setEmpty(false);
    setError("No se han podido cargar los datos. Inténtalo de nuevo.");
    console.error(error);

  } finally {
    setLoading(false);
  }
}

botonRecarga.addEventListener("click", cargarDatos);
```

async/await no bloquea el navegador

- await pausa la función, pero el hilo principal sigue activo.
- El Event Loop continúa procesando eventos y render.

Validación explícita del contrato

- fetch no lanza error automáticamente en 404/500 → se valida response.ok.
- Se comprueba que la estructura es la esperada (Array.isArray).

Esto es **programación defensiva**.

Render eficiente con DocumentFragment

- Construcción en memoria
- Una sola inserción en el DOM real
- Menos reflows

Gestión clara de estados

- Loading siempre visible cuando corresponde
- Error centralizado
- Empty diferenciado de Error
- finally garantiza limpieza visual

Event delegation (escalabilidad)

```
lista.addEventListener("click", (e) => {
  const elemento = e.target.closest(".item");
  if (!elemento) return;

  const id = elemento.dataset.id;
  console.log("Elemento pulsado:", id);
```

```
});
```

Ventajas:

- Un solo listener
- Funciona con elementos creados dinámicamente
- Escala mejor que un listener por elemento

¿Qué es Vanilla JS?

Vanilla JS significa JavaScript puro, sin frameworks ni librerías externas.

Es simplemente JavaScript estándar del navegador

```
document.querySelector("button").addEventListener("click", () => {
  console.log("Click");
});
```

Sin usar:

- React
- Vue.js
- Angular
- jQuery

¿Qué incluye Vanilla JS?

Todo lo que ofrece el navegador de forma estándar:

- DOM API
- Fetch API
- Web Storage
- Events
- Promesas
- async/await
- Modules ES6
- Web APIs HTML5

Es decir, **no es limitado**, es completo.

Ejemplo comparativo

Con Vanilla JS

```
const lista = document.getElementById("lista");
lista.innerHTML = "<li>Elemento</li>";
```

Con React

```
function Lista() {
  return <li>Elemento</li>;
```

}

En React no manipulas directamente el DOM.
En Vanilla JS sí.

Ventajas de Vanilla JS

Sin dependencias externas. No necesitas instalar nada.

Menor tamaño de proyecto. No hay librerías pesadas.

Control total del DOM. Sabes exactamente qué ocurre.

Mejor comprensión del navegador

Desventajas

Más código manual

Tienes que gestionar: Estado, Render, Eventos, Arquitectura

Puede volverse complejo en proyectos grandes, por eso existen frameworks.

¿Por qué usar Vanilla JS en vuestra práctica?

Porque permite evaluar:

- Uso eficiente del DOM
- Programación JS clara y modular
- Comprensión del Event Loop
- Control de asincronía
- Optimización real

Si usas un framework, parte del trabajo lo hace la herramienta. Con Vanilla JS, el criterio técnico es visible.

JSON

JSON: el formato dominante

- Texto legible, fácil de parsear
- Mapea bien a objetos JS
- Ideal para APIs REST

- Común en frontend moderno

JSON (JavaScript Object Notation) es:
- Un formato de intercambio de datos ligero, basado en texto, estructurado en pares clave-valor.
- Es independiente del lenguaje, aunque su sintaxis está inspirada en objetos de JavaScript

JSON: estructura y tipado

Estructura básica de un JSON
Un JSON válido solo puede contener:
- Un **objeto**
- O un **array**

Ejemplo objeto:
```
{
  "nombre": "Museo Es Baluard",
  "ciudad": "Palma"
}
```
Ejemplo array:
```
[
  {
    "nombre": "Evento 1"
  },
  {
    "nombre": "Evento 2"
  }
]
```

Tipos de datos permitidos en JSON
JSON tiene un tipado muy limitado y estricto.
Tipos válidos:
String
"nombre": "Mallorca"
Number
"precio": 12.5
Boolean
"activo": true
Null
"descripcion": null
Object
"ubicacion": {
  "lat": 39.5696,
  "lng": 2.6502
}
Array
"tags": ["arte", "cultura", "museo"]

Reglas estrictas de sintaxis
JSON es más estricto que JavaScript.
- Claves siempre entre comillas dobles
- Strings siempre con comillas dobles
- No admite comentarios

```
Correcto:
{
  "nombre": "Evento",
  "año": 2024
}
Incorrecto:
{
  nombre: "Evento",
}
```

Cómo se construye un JSON

Definir el modelo de datos
Antes de escribir JSON, hay que definir:

- Qué entidades existen
- Qué atributos tienen
- Qué relaciones hay entre ellas

Ejemplo: Web-App cultural
Entidad “Evento”:

- id
- nombre
- fecha
- isla
- ubicación
- imagen
- categoría

Crear estructura coherente
Ejemplo estructurado:

```
[
  {
    "id": 1,
    "nombre": "Festival de Música",
    "fecha": "2026-07-15",
    "isla": "Mallorca",
    "ubicacion": {
      "lat": 39.5696,
      "lng": 2.6502
    },
    "categoria": "Música",
    "imagen": "festival.jpg"
  }
]
```

¿Por qué usar schema.org para crear el JSON?

Qué es schema.org

Schema.org es una iniciativa impulsada por:

- Google
- Microsoft
- Yahoo
- Yandex

Su objetivo es definir vocabularios estándar para describir datos estructurados en la Web.

Estandarización semántica

Si cada grupo define su propio modelo:

```
{
  "nombreEvento": "...",
  "fechaEvento": "...",
  "localizacion": "..."
}
```

No existe garantía de interoperabilidad.
Con schema.org:

```
{
  "@type": "Event",
  "name": "...",
  "startDate": "...",
  "location": {...}
}
```

Se usa un modelo reconocido globalmente -> Reduce ambigüedad semántica.

Interoperabilidad

Schema.org permite que los datos puedan ser:

- Interpretados por buscadores
- Integrados en otras plataformas
- Reutilizados por otros sistemas
- Compartidos entre grupos

Esto es clave en proyectos colaborativos.

SEO y visibilidad

Los motores de búsqueda interpretan directamente schema.org.
Ejemplo:

- Eventos culturales
- Museos
- Exposiciones
- Localizaciones

Si se estructura correctamente, aporta valor en Web Semántica y SEO.

Modelo formal de datos

Schema.org obliga a pensar en:

- Tipos (`Event`, `Place`, `Organization`)
- Propiedades formales (`name`, `startDate`, `geo`)
- Relaciones entre entidades

Ejemplo estructurado:

```
{
  "@context": "https://schema.org",
  "@type": "Event",
  "name": "Festival de Música",
  "startDate": "2026-07-15",
  "location": {
    "@type": "Place",
    "name": "Auditorio de Palma",
    "geo": {
      "@type": "GeoCoordinates",
      "latitude": 39.5696,
      "longitude": 2.6502
    }
  }
}
```

Esto no es solo JSON.
Es un modelo ontológico.

Escalabilidad y coherencia

Si en las prácticas de TM hay:

- Museos
- Eventos
- Exposiciones
- Artistas

Schema.org ya define tipos para todos:

- `Event`
- `Place`
- `Museum`
- `Person`
- `CreativeWork`

Evita que cada grupo invente estructuras distintas.

Enfoque de ingeniería en la práctica TM

Usar schema.org implica:
- Diseñar modelo antes que implementación
- Separar datos de presentación
- Aplicar Web Semántica
- Pensar en interoperabilidad
Es una decisión arquitectónica, no estética.

Relación con la práctica

En los criterios se evalúa:

- Uso correcto de JSON
- Conformidad con esquema semántico
- Web Semántica
- Integración de datos externos

Schema.org permite cumplir todos esos puntos de forma estructurada.

Usar schema.org no es obligatorio técnicamente (si para TM), porque:

- Estandariza el modelo
- Mejora interoperabilidad
- Refuerza SEO
- Aporta rigor semántico
- Eleva el nivel técnico del proyecto

Desde nuestra perspectiva: No solo construimos una Web-App, construimos datos comprensibles por máquinas.

Ejemplo completo

Características técnicas

- `@context` define el vocabulario semántico.
- `@graph` permite incluir múltiples entidades.
- Cada monumento tiene:
    - `@type`
    - `@id como URI, permite identificar cada recurso de forma inequívoca.`
    - `name`
    - `geo`
- `geo` usa el tipo `GeoCoordinates`.

```
{
  "@context": "https://schema.org",
  "@graph": [
    {
      "@id": "https://ejemplo.org/monumentos/la-seu",
      "@type": "Place",
      "name": "La Seu - Catedral de Mallorca",
      "geo": {
        "@type": "GeoCoordinates",
        "latitude": 39.5677,
        "longitude": 2.6483
      }
    },
    {
```

```
      "@id": "https://ejemplo.org/monumentos/castell-bellver",
      "@type": "Place",
      "name": "Castell de Bellver",
      "geo": {
        "@type": "GeoCoordinates",
        "latitude": 39.5636,
        "longitude": 2.6204
      }
    },
    {
      "@id": "https://ejemplo.org/monumentos/torre-del-oro",
      "@type": "Place",
      "name": "Torre del Oro",
      "geo": {
        "@type": "GeoCoordinates",
        "latitude": 37.3824,
        "longitude": -5.9963
      }
    }
  ]
}
```

Cómo validar schema.org con Google Structured Data Testing

La antigua herramienta: Structured Data Testing Tool, es reemplazada por:

- **Rich Results Test** (para SEO en Google)
- **Schema Markup Validator** (para validación estándar schema.org)

Opción 1 — Validar con Rich Results Test (Google)

Cuando quieres saber si tu JSON-LD:

- Es válido para Google
- Genera rich snippets
- Cumple requisitos SEO

https://search.google.com/test/rich-results

La herramienta mostrará:

- Tipos detectados (Event, Place…)
- Propiedades válidas
- Advertencias
- Errores críticos

Opción 2 — Validar con Schema Markup Validator

Herramienta oficial de Schema.org

https://validator.schema.org/
Esta herramienta valida:

- Sintaxis
- Uso correcto de tipos
- Propiedades compatibles

No evalúa SEO específico de Google.

Validar schema.org no es solo comprobar sintaxis.
Es asegurar que:

- Los datos son comprensibles por máquinas
- Cumplen un estándar internacional
- Pueden ser reutilizados y posicionados

Desde ingeniería:
Diseñar datos bien estructurados es tan importante como programar bien.

Diferencia entre JSON y Objeto JavaScript

Aunque parecen iguales, **no son lo mismo**.

- JSON → Formato de datos en texto.
- Objeto JS → Estructura de datos en memoria.

JSON	Objeto JS
Texto plano	Estructura en memoria
Formato de intercambio	Tipo de dato del lenguaje
Se transmite por red	Se ejecuta en el motor JS

Ejemplo JSON (texto):

```
{
  "nombre": "Museo",
  "activo": true
}
```

Objeto JavaScript:

```
{
  nombre: "Museo",
  activo: true
}
```

Tipos permitidos

JSON permite:

- string
- number
- boolean
- null
- object
- array

Objeto JS permite además:

- funciones
- undefined
- Date
- Map
- Set
- símbolos
- métodos

Ejemplo válido en JS pero inválido en JSON:

```
{
  nombre: "Evento",
  fecha: new Date(),
  metodo() { console.log("hola"); }
}
```

Conversión entre ambos

De JSON a objeto JS

```
const obj = JSON.parse(jsonString);
```

Convierte texto JSON en objeto JS.

De objeto JS a JSON

```
const json = JSON.stringify(obj);
```

Convierte objeto JS en texto JSON.

Uso en Web-Apps

Flujo típico:

1. Servidor envía JSON (texto)
2. `fetch()` lo recibe
3. `response.json()` lo convierte en objeto JS
4. Se trabaja con él en memoria

Cómo incluir fotos, audio y vídeo en JSON-LD (schema.org)

1) Elegir la entidad principal

Si estás describiendo un monumento, lo habitual es usar un tipo derivado de `Place`, por ejemplo:

- `Place`
- `LandmarksOrHistoricalBuildings` (recomendado para monumentos)

2) Representar cada media como un objeto schema.org

En schema.org, los media se describen como:

- **Foto** → `ImageObject`
- **Audio** → `AudioObject`
- **Vídeo** → `VideoObject`

Cada uno debería tener, como mínimo:

- `contentUrl` (URL del archivo)
  y opcionalmente:
- `encodingFormat` (MIME: `image/jpeg`, `audio/mpeg`, `video/mp4`)
- `name`, `description`, `uploadDate`
- `duration` (para audio/vídeo en formato ISO 8601, ej. `PT2M30S`)
- `thumbnailUrl` (para vídeo)

3) Vincular los media al monumento

Para un `Place`, una forma estándar y muy compatible es usar **`subjectOf`** (propiedad de `Thing`) para asociar recursos (fotos, audio-guías, vídeos, etc.).

Una alternativa para imágenes es : `image` para fotos (permite URL o `ImageObject`)

4) Reglas de "validación" típicas (evitar errores)

- Fechas en ISO: `"uploadDate": "2026-02-26"`
- Duración en ISO 8601: `"duration": "PT1M45S"`
- URLs absolutas recomendables (mejor que rutas relativas)
- No añadir claves no estándar (p. ej. `"foto1"`, `"mp3"`, etc.)

```
{
  "@context": "https://schema.org",
  "@type": "LandmarksOrHistoricalBuildings",
  "@id": "https://ejemplo.org/monumentos/castell-bellver",
  "name": "Castell de Bellver",
  "geo": {
    "@type": "GeoCoordinates",
    "latitude": 39.5636,
    "longitude": 2.6204
  },
  "image": [
    {
      "@type": "ImageObject",
      "@id": "https://ejemplo.org/media/castell-bellver/foto-1",
```

```
      "name": "Castell de Bellver (fachada)",
      "contentUrl": "https://ejemplo.org/media/castell-bellver/bellver-01.jpg",
      "encodingFormat": "image/jpeg"
    },
    {
      "@type": "ImageObject",
      "@id": "https://ejemplo.org/media/castell-bellver/foto-2",
      "name": "Patio interior",
      "contentUrl": "https://ejemplo.org/media/castell-bellver/bellver-02.jpg",
      "encodingFormat": "image/jpeg"
    }
  ],
  "subjectOf": [
    {
      "@type": "AudioObject",
      "@id": "https://ejemplo.org/media/castell-bellver/audio-guia-es",
      "name": "Audioguía (ES) — Castell de Bellver",
      "contentUrl": "https://ejemplo.org/media/castell-bellver/bellver-es.mp3",
      "encodingFormat": "audio/mpeg",
      "duration": "PT2M10S",
      "uploadDate": "2026-02-26"
    },
    {
      "@type": "VideoObject",
      "@id": "https://ejemplo.org/media/castell-bellver/video-presentacion",
      "name": "Vídeo presentación — Castell de Bellver",
      "contentUrl": "https://ejemplo.org/media/castell-bellver/bellver.mp4",
      "encodingFormat": "video/mp4",
      "duration": "PT1M20S",
      "thumbnailUrl": "https://ejemplo.org/media/castell-bellver/bellver-thumb.jpg",
      "uploadDate": "2026-02-26"
    }
  ]
}
```


¿Qué ocurre realmente cuando haces `fetch()`?

```
const response = await fetch("/datos.json");
const data = await response.json();
```

- Se descarga el archivo JSON como texto
- `response.json()` lo parsea
- Se convierte en un **objeto JavaScript en memoria**

En ese momento todo el JSON está cargado en RAM como estructura de objetos.

Se parsea el contenido completo antes de devolverte el objeto.
Para archivos pequeños/medianos (práctica académica):

- Es totalmente correcto
- Es lo habitual
- Es simple y mantenible

Puede ser problemático si tiene miles o millones de registros. Es muy pesado (decenas de MB)

Transparencia 12 — Fetch: firma y retorno

```
fetch(url, options) -> Promise<Response>
```

`Response` incluye:

- `status`, `ok`
- `headers`
- `json()`, `text()`

Clave: fetch solo rechaza en errores de red, no en 404/500 (eso lo haces tú).

Función robusta: fetch JSON con control de status

```
async function fetchJson(url) {
  const res = await fetch(url, { headers: { "Accept": "application/json" } });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

Manejo de errores: taxonomía útil

Tipos de fallo:

1. red / offline
2. servidor (5xx)
3. recurso no encontrado (404)
4. datos inválidos (JSON mal formado)
5. timeout (respuesta tarda demasiado)
6. rate limit (429) en APIs externas

UX: cada uno debería tener mensaje y acción sugerida.

UI state machine (loading/success/empty/error)

Patrón obligado para Web-Apps:

- `state.status = "loading" | "success" | "empty" | "error"`
- render según estado

Evaluación: estados y mensajes de error explicativos se valoran

Ejemplo: render según status

```
function render(state) {
  if (state.status === "loading") return renderLoading();
  if (state.status === "error") return renderError(state.errorMsg);
```

```
  if (state.items.length === 0) return renderEmpty();
  return renderList(state.items);
}
```

Mini-caso completo: cargar catálogo remoto

Objetivo:

- cargar dataset.json desde `/data/dataset.json` o URL
- mostrar loading
- render lista
- soportar error y reintento
- fallback a cache (opcional)

Arquitectura:

- `loadItems()` asíncrona
- `state.status`
- `render(state)`

Ejemplo: loadItems (robusto)

```
async function loadItems() {
  state.status = "loading";
  render(state);

  try {
    const res = await fetchWithTimeout("./data/dataset.json", 8000);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);

    const raw = await res.json();
    state.items = normalizeItems(raw);
    state.status = state.items.length ? "success" : "empty";
  } catch (err) {
    state.status = "error";
    state.errorMsg = err.name === "AbortError"
      ? "La carga tardó demasiado. Reintenta."
      : "No se pudo cargar el catálogo. Revisa tu conexión y reintenta.";
  }

  render(state);
}
```